

Quantization.h & Quantization.c

A Beginner's Complete Guide

Types, configs, structs, and initialisation functions — every concept explained

What you will learn:

- What quantization is and why it matters for on-device training
- Why floating-point is expensive on a microcontroller
- The six quantization types (qtype_t enum) explained with examples
- The three QConfig structs — what each field means and why it exists
- The quantization_t wrapper struct and the void* pointer concept
- Every init function explained line by line
- How all pieces connect together in the full system
- A complete worked example: quantizing a weight tensor

1 — What Is Quantization and Why Does It Exist?

A neural network stores its weights as floating-point numbers — typically 32-bit floats. A single float uses 4 bytes of memory and requires a floating-point unit (FPU) to compute. On a powerful PC this is fine. On the STM32 Nucleo-F746ZG with only 320 KB of RAM and a limited FPU, storing and computing with thousands of 32-bit floats is slow and quickly exhausts memory.

Quantization is the process of converting those 32-bit floats into smaller integer representations — for example, 8-bit integers. Instead of storing the exact float value, you store a compressed integer and a small number called a **scale** that lets you recover an approximation of the original value.

The core idea with a concrete example:

```
// Original float weight:
float w = 0.382f;

// After quantization to 8-bit SYM_INT32:
scale = 0.003f; // how much each integer step is worth
mantissa = 127; // stored as int32 (fits in int8 range)

// To recover the approximate float:
recovered = mantissa * scale = 127 * 0.003 = 0.381f
// Error: |0.382 - 0.381| = 0.001 ← tiny, acceptable
```

★ **Memory saving:** A float32 weight uses 4 bytes. An int8 weight uses 1 byte — 4× smaller. A 1000-weight model shrinks from 4 KB to 1 KB, freeing precious SRAM for activations and gradients.

Why integers are faster on the STM32:

Operation	Float32	Int8/Int32
Addition	Multiple cycles, needs FPU	Single cycle, uses ALU
Multiplication	Multiple cycles, needs FPU	1-2 cycles on ARM Cortex-M7
Memory per value	4 bytes	1 byte (int8) or 4 bytes (int32)
Hardware needed	Floating-Point Unit (FPU)	Integer ALU (always present)

2 — The qtype_t Enum — Six Quantization Types

An **enum** (enumeration) is a way of giving names to a set of integer constants. Instead of remembering that type 0 means INT32 and type 2 means SYM_INT32, you write the name directly. The compiler replaces the names with numbers.

```
typedef enum qtype {
    INT32, // = 0
    FLOAT32, // = 1
    SYM_INT32, // = 2
    SYM, // = 3
    ASYM, // = 4
    BOOL // = 5
} qtype_t;
```

typedef enum — creates a new type called `qtype_t`. The compiler automatically assigns integer values starting from 0. You write `SYM_INT32` in your code; the compiler sees 2. This is why `conversionMatrix[6][6]` has exactly 6 rows and 6 columns — one per enum value.

All six types explained:

Type	Bytes/element	Has scale?	Has zeroPoint?	Used for
INT32	4	No	No	Raw integer accumulation, intermediate results
FLOAT32	4	No	No	Training weights, activations — full precision
SYM_INT32	4	Yes	No	FQT training — int32 mantissa + float scale
SYM	N/8	Yes	No	General N-bit symmetric quantization
ASYM	N/8	Yes	Yes	N-bit asymmetric — scale + zeroPoint offset
BOOL	1/8	No	No	Sparsity masks — 1 bit per element, 8 per byte

■ **Symmetric vs Asymmetric:** Symmetric quantization maps the range $[-\text{max}, +\text{max}]$ to integers centred at zero. Asymmetric quantization maps any range $[\text{min}, \text{max}]$ — it adds a zeroPoint offset so that 0.0f does not have to map to integer 0. Symmetric is simpler and faster; asymmetric is more flexible.

3 — The Three QConfig Structs

Three of the six quantization types carry extra configuration parameters — scale, number of bits, zeroPoint, rounding mode. These are stored in dedicated structs called QConfigs. INT32, FLOAT32, and BOOL need no extra config — their format is fixed.

Struct 1 — symInt32QConfig_t (used by SYM_INT32):

symInt32QConfig_t		
float	scale	How much one integer step is worth in real units. $\text{real_value} = \text{mantissa} \times \text{scale}$. Computed from the data range during quantization.
roundingMode_t	roundingMode	Which rounding strategy to use when converting float $\rightarrow$ int. Either HALF_AWAY (standard) or SR_HALF_AWAY (stochastic).
uint8_t	qMaxBits	The effective bit-width of the quantized values. Default is 16. If qMaxBits=8, values fit in $[-128, 127]$. Controls how fine-grained the quantization is.

How the fields work together:

```
// qMaxBits = 8:
qMax = powf(2, qMaxBits-1) - 1 = 127
qMin = -powf(2, qMaxBits-1) = -128

// scale is derived from the actual data:
scale = absMax / qMax // e.g. 0.5 / 127 = 0.00394

// quantize:
mantissa = clamp(float_val / scale, qMin, qMax)
// e.g. 0.382 / 0.00394 = 96.9  $\rightarrow$  rounds to 97

// dequantize:
recovered = mantissa * scale = 97 * 0.00394 = 0.382f
```

Struct 2 — symQConfig_t (used by SYM):

symQConfig_t		
float	scale	Same as SYM_INT32 — how much one step is worth.
uint8_t	qBits	Exact bit-width of stored values (e.g. 4, 8, 16). Unlike SYM_INT32 (which always stores int32), SYM packs values into exactly qBits bits in memory.
roundingMode_t	roundingMode	Rounding strategy — HALF_AWAY or SR_HALF_AWAY.

■ **SYM vs SYM_INT32:** Both are symmetric. The difference is storage. SYM_INT32 always stores mantissas as full int32 (4 bytes each) — simple to index and compute with, but uses more memory. SYM packs values into exactly qBits bits — more compact but requires the byteConversion() function to read/write individual values.

Struct 3 — asymQConfig_t (used by ASYM):

asymQConfig_t		
float	scale	How much one integer step is worth in real units.
int16_t	zeroPoint	Which integer value represents real 0.0f. In symmetric quantization zeroPoint is always 0. In asymmetric it shifts the range to cover [min, max].
uint8_t	qBits	Bit-width of stored values.
roundingMode_t	roundingMode	Rounding strategy.

Why does ASYM need a zeroPoint?

Imagine weights in the range [0.1, 0.9] — all positive. Symmetric quantization wastes half its range on negative values that do not exist. Asymmetric shifts the integer range so it covers exactly [0.1, 0.9]:

```
// ASYM example: weights in [0.1, 0.9], qBits=8
qMax = 255 (2^8 - 1)
scale = (0.9 - 0.1) / 255 = 0.00314
zeroPoint = round(0.1 / 0.00314) = 32

// quantize: q = round(x / scale) - zeroPoint
// dequantize: x = (q + zeroPoint) * scale

// e.g. x=0.5:
q = round(0.5 / 0.00314) - 32 = 159 - 32 = 127
recovered = (127 + 32) * 0.00314 = 0.500f ✓
```

4 — The quantization_t Wrapper Struct

The three QConfig structs are different sizes and shapes. A tensor needs to carry its quantization info around without the tensor struct knowing in advance which type it will be. The solution is a wrapper struct that holds two things: the type tag, and a generic pointer to whichever config struct applies.

```
typedef struct quantization {
    qtype_t type;
    void *qConfig;
} quantization_t;
```

type — which of the 6 quantization types this is. Acts as a tag that tells you how to interpret qConfig. **void *qConfig** — a pointer to the config struct, OR NULL if no config is needed (INT32, FLOAT32, BOOL). void* is a 'pointer to anything' — it accepts any struct pointer without a type mismatch error.

What is void* — pointer to anything:

Normally a pointer has a specific type — `int32_t *` points to an int32, `float *` points to a float. A `void *` is a special pointer with no type attached — it just holds an address. You can store any pointer in it, and cast it back to the correct type when you need to use it.

```
symInt32QConfig_t myConfig;
quantization_t q;
q.type = SYM_INT32;
q.qConfig = &myConfig; // float* stored as void* — no error

// To use it, cast back:
symInt32QConfig_t *cfg = (symInt32QConfig_t *)q.qConfig;
float scale = cfg->scale;
```

■ **Why not use a union instead?** A union would fix the size at compile time to the largest member. void* is more flexible — the config struct lives wherever you allocated it, and quantization_t just holds its address. The caller must know the type (from the type field) before casting.

The full picture of quantization_t:

```
quantization_t
■■■ type = SYM_INT32 (enum value = 2)
■■■ qConfig ■■■■■■■■■■ → symInt32QConfig_t
■■■ scale = 0.00394f
■■■ roundingMode= HALF_AWAY
■■■ qMaxBits = 8

quantization_t (for FLOAT32):
■■■ type = FLOAT32
■■■ qConfig = NULL ← no config needed
```

5 — Init Functions for QConfig Structs

Rather than setting every field manually each time, the code provides init functions that fill a config struct with safe default values. You pass in the parameters you care about; the function handles the rest.

initSymInt32QConfig() — default qMaxBits=16:

```
void initSymInt32QConfig(
    roundingMode_t roundingMode,
    symInt32QConfig_t *symInt32QConfig) {
    symInt32QConfig->roundingMode = roundingMode;
    symInt32QConfig->scale = 1.f;
    symInt32QConfig->qMaxBits = 16;
}
```

roundingMode — caller decides HALF_AWAY or SR_HALF_AWAY. **scale = 1.f** — default scale is 1. This means mantissa == real value initially. The actual scale is computed and written later when `convertFloatTensorToSymInt32Tensor()` runs. **qMaxBits = 16** — default. $2^{(16-1)-1} = 32767$ range. Provides good precision before a specific bit-width is chosen. ***symInt32QConfig** — pointer to the struct to fill. The function modifies the struct in place via the pointer.

initSymInt32QConfigWithQMaxBits() — custom bit-width:

```
void initSymInt32QConfigWithQMaxBits(
    roundingMode_t roundingMode,
    symInt32QConfig_t *symInt32QConfig,
    uint8_t qMaxBits) {
    symInt32QConfig->roundingMode = roundingMode;
    symInt32QConfig->scale = 1.f;
    symInt32QConfig->qMaxBits = qMaxBits;
}
```

Same as `initSymInt32QConfig` but lets the caller specify `qMaxBits`. Use this when you want 8-bit precision: pass `qMaxBits=8` → range `[-128, 127]`. Or 4-bit: `qMaxBits=4` → range `[-8, 7]`. More compression, less precision.

initSymQConfig():

```
void initSymQConfig(uint8_t qBits,
    roundingMode_t roundingMode,
    symQConfig_t *symQConfig) {
    symQConfig->qBits = qBits;
    symQConfig->roundingMode= roundingMode;
    symQConfig->scale = 1.f;
}
```

Initialises a SYM config. `qBits` sets the actual storage bit-width. `scale=1.f` again as a safe default — the real scale is computed when quantizing.

initAsymQConfig():

```
void initAsymQConfig(uint8_t qBits,
    roundingMode_t roundingMode,
    asymQConfig_t *asymQConfig) {
    asymQConfig->qBits = qBits;
    asymQConfig->roundingMode= roundingMode;
    asymQConfig->scale = 1.f;
    asymQConfig->zeroPoint = (uint16_t)0;
}
```

Same pattern but also initialises `zeroPoint` to 0. `zeroPoint=0` means symmetric initially — the asymmetric offset is computed during the actual quantization conversion. Note: the comment in the code casts to `uint16_t` but `zeroPoint` is declared as `int16_t` — a minor inconsistency; it still works because 0 is the same in both.

6 — Init Functions for quantization_t

These six functions fill a `quantization_t` struct — setting the type tag and pointing `qConfig` to the right config struct (or NULL).

Simple types — no config needed:

INT32, FLOAT32, and BOOL have fixed formats — no scale, no bit-width choice, no zeroPoint.

qConfig = NULL — null pointer means 'no config'. Code that reads qConfig always checks the type first and only dereferences qConfig when the type requires it. Three separate functions instead of one because it keeps call sites readable:

initFloat32Quantization(&q;) is clearer than
setQuantType(&q;, FLOAT32).

Each function takes two pointers: 1. The config struct already filled by `initSymInt32QConfig()` etc.

2. The `quantization_t` to wire up.

`quantization->qConfig = symInt32QConfig` — stores the address of the config struct into the `void*` field. Now `quantization_t` points to its config.

Important: the config struct must stay alive as long as the `quantization_t` is used — they are separate objects; `quantization_t` only holds a pointer to the config, not a copy.

symInt32QConfig_t	Config for SYM_INT32: scale + roundingMode + qMaxBits
symQConfig_t	Config for SYM: scale + qBits + roundingMode
asymQConfig_t	Config for ASYM: scale + zeroPoint + qBits + roundingMode
quantization_t	Wrapper: type tag + void* pointer to one of the three configs
void*	Pointer to anything — holds address of any config struct
NULL qConfig	Used for INT32, FLOAT32, BOOL — no config needed
scale = 1.f init	Default safe value — real scale computed during conversion
SYM_INT32 formula	$\text{mantissa} = \text{clamp}(x / \text{scale}, q_{\text{Min}}, q_{\text{Max}})$
ASYM formula	$q = \text{round}(x/\text{scale}) - \text{zeroPoint} \mid x = (q + \text{zeroPoint}) * \text{scale}$